Programming paradigms: procedural and object-oriented

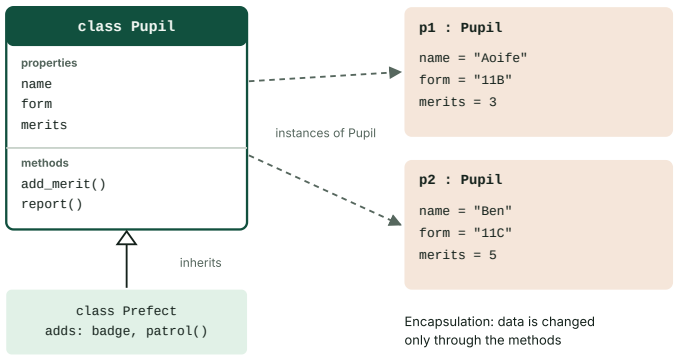
D01 FACT FILE

Specification: students should be able to

- Describe the following programming paradigms: procedural programming and object-oriented programming

CCEA fact file: Software Development Trends

Key words: paradigm, procedural, imperative, sequential, procedure, function, object-oriented, object, class, property, method, inheritance, encapsulation



A Pupil class, two objects created from it, and a Prefect class that inherits from it

Procedural programming

In procedural programming the programmer gives the computer a sequence of instructions to follow, one after another. The language is imperative, meaning each statement is a command. Larger programs are broken down (top-down design) into procedures and functions, which are named blocks of code that carry out one task and can be called from anywhere in the program. Data is usually held in variables that are passed into these procedures.

Object-oriented programming

In object-oriented programming the solution is designed around objects. A class is a template describing what every object of that type will know (its properties or attributes) and do (its methods). An object is one instance created from the class, with its own values. Objects communicate by calling each other's methods. Encapsulation keeps an object's data inside it so it can only be changed through its methods. Inheritance lets a new class reuse and extend an existing class. Because classes are self-contained they are easier to test and can be reused in other programs.

T Key terms from the mark schemes

Procedural programming
Step-by-step instructions executed in sequence, organised into procedures and functions.
Credited: U4 2019 Q7, 2024 Q5, 2025 Q7

Object-oriented programming
Programs built from self-contained objects that combine data (properties) and methods.
Credited: U4 2024 Q5, 2025 Q7

Class
A blueprint defining the properties and methods of objects.
Credited: U4 2025 Q7

Object
An instance created from a class with its own data.
Credited: U4 2025 Q7

Encapsulation
Keeping an object's data inside it, accessed only through its methods.
Credited: U4 2025 Q7

Inheritance
A class taking on the properties and methods of another class.
Credited: U4 2025 Q7

C C# example

Procedural programming

```
static double Area(double width, double height)
{
    return width * height;
}

Console.Write("Width: ");
double w = Convert.ToDouble(Console.ReadLine());
```

```
Console.Write("Height: ");
double h = Convert.ToDouble(Console.ReadLine());
Console.WriteLine("Area is " + Area(w, h));
```

Exam focus

Expect fill-the-gap questions on paradigm vocabulary (2024 Q5(a)) and a QWC description of both approaches (2025 Q7). Write in paragraphs, cover both paradigms in similar depth, and use the technical terms – generic answers stay in Level 1.

Unit 4 2025 Q7	[6]	Procedural vs object-oriented programming (QWC)
Unit 4 2024 Q5	[9]	Paradigms fill-in; compilers and interpreters true/false

? Check yourself

1. Describe procedural programming. [3]
2. State the difference between a class and an object. [2]
3. In OOP, what is meant by a property and a method? [2]

Answers: 1. Any three from: the programmer writes step-by-step instructions telling the computer exactly what to do (1) / it is imperative – each statement is a command (1) / instructions are executed sequentially, one after another, so the order is the program flow (1) / code is grouped into procedures and functions that can be called when needed (1) / the program is designed top-down – the main task is broken into smaller sub-tasks (1) / data is held in variables, kept separate from the code, and passed into procedures (1) 2. A class is a blueprint/template that defines the properties and methods for a type of object (1) / an object is a single instance created from a class, with its own data values (1). Accept e.g. class Pupil; objects p1 and p2. 3. A property (attribute) is an item of data that an object holds, e.g. a Pupil's Name or Form (1) / a method is an action/subroutine an object can carry out, e.g. AddMerit() (1)